TEMA 6.- EL LENGUAJE DML

1 Sentencias DML

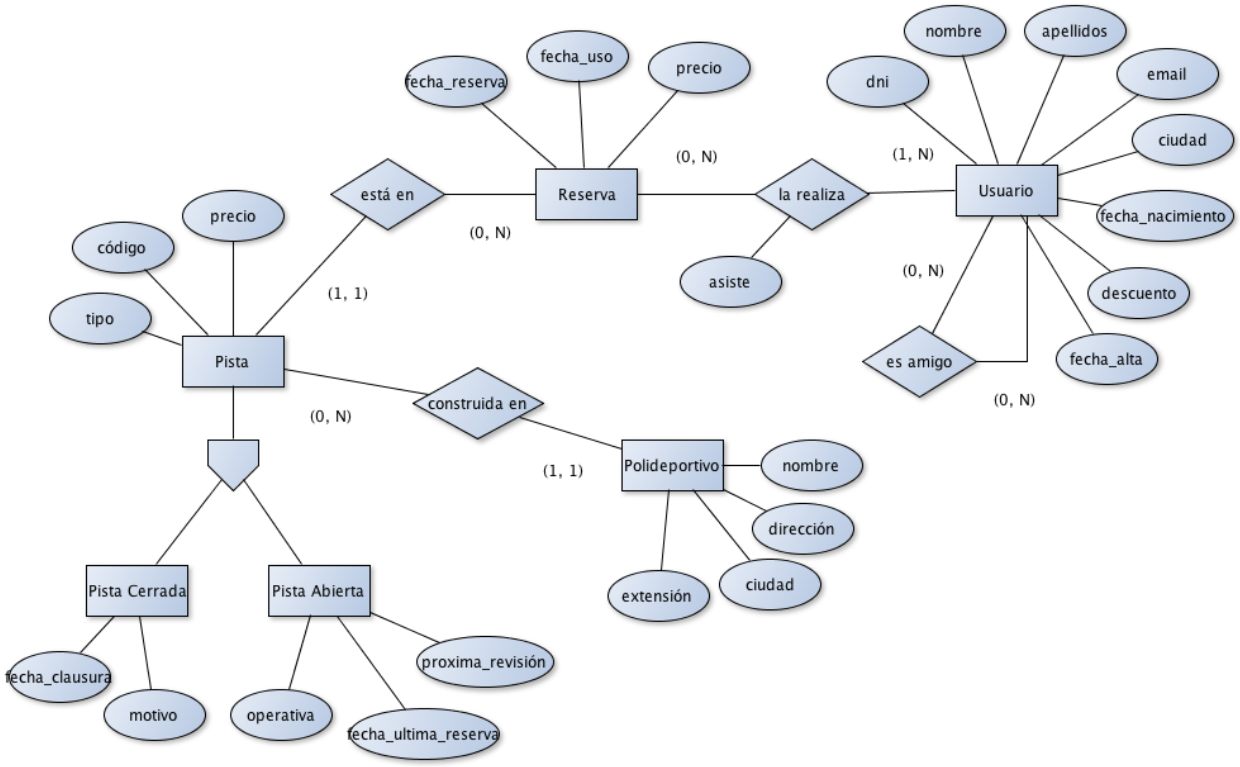
En el lenguaje SQL existen 4 sentencias que forman el DML (Data Manipulation Language, Lenguaje de Manipulación de Datos). Son aquellas sentencias que nos permiten manipular la información que almacenamos en las Bases de Datos.

Existen 4 instrucciones que nos permitirán insertar datos en una tabla (`INSERT`), modificar esos datos (UPDATE), eliminarlos (DELETE) y consultarlos (SELECT).

2 Base de Datos de prueba (reservas)

Todos los ejemplos de este bloque se han escrito tomando como referencia la Base de Datos `reservas.`

A continuación, se muestran el modelo entidad-relación y relacional de esta Base de Datos de prueba:




```
pistas (id, codigo, tipo, precio, id_polideportivo)

pistas_abiertas(id_pista, operativa, precio, fecha_ultima_reserva,
proxima_revision)

pistas_cerradas (id_pista, fecha_clausura, motivo)

polideportivos (id, nombre, direccion, ciudad, extension)

reservas (id, fecha_reserva, fecha_uso, precio, -id_pista)

usuarios (id, dni, nombre, apellidos, email, ciudad, fecha_nacimiento, descuento,
fecha_alta)
```

```
usuario_usuario (id_usuario, id_amigo)

usuario_reserva (id_usuario, id_reserva, asiste)

Los campos en negrita son claves primarias y los campos en cursiva son claves
foráneas.
```

3 Modificar Información

3.1 Inserción de registros

La inserción de nuevos registros a una tabla se efectúa con la sentencia INSERT, que tiene el siguiente formato:

```
INSERT INTO nombre_tabla [ '('columnas')' ]

{ VALUES '(' { valores } ')',} | consulta
```

Veamos algunos ejemplos:

- Para insertar una fila en una tabla

```
INSERT INTO usuarios (id, dni, nombre, apellidos, email, ciudad, descuento,
fecha_nacimiento, fecha_alta)

VALUES (100, '12345678A', 'Antonio', 'García', 'agarcia@gmail.com', 'Málaga', 0.25,
'1990-12-12', '2020-12-04');

-- Como se inserta una fila con todos los valores de los campos, no haría falta
especificar los nombres de los campos. Aunque el orden de los valores debe
coincidir con el orden que tienen los campos cuando se creó la tabla.

INSERT INTO usuarios

VALUES (200, '12345677A', 'Miguel', 'García', 'magarcia@gmail.com', 'Zaragoza',
'1990-12-12', 0.3, '2003-02-01');

-- Inserta una fila con todos los valores de los campos, excepto el campo fecha de
nacimiento que no es obligatorio.

INSERT INTO usuarios (id, dni, nombre, apellidos, email, ciudad, descuento,
fecha_alta)

VALUES (45, '12345679A', 'Luis', 'García', 'lgarcia@gmail.com', 'Málaga', 0.15,
'1990-12-12');
```

- Para insertar varias filas en una tabla

```
INSERT INTO usuarios (id, dni, nombre, apellidos, email, ciudad, descuento,
fecha_alta)

VALUES (300, '12345678A', 'Pepe', 'Sanz', 'psanz@gmail.com', 'Málaga', 0.25, '1990-
12-12'),

(301, '98765432Z', 'Luis', 'Peréz', 'lperez@gmail.com', 'Málaga', 0.25, '1988-01-
03');
```

Para insertar el resultado de una consulta en una tabla (suponiendo que la tabla `otros_usuarios` existe y tiene al menos los campos indicados del mismo tipo que en la tabla `usuarios`)

```
INSERT INTO otros_usuarios (id, dni, nombre, apellidos, email, descuento,
fecha_alta)

SELECT id, dni, nombre, apellidos, email, descuento, fecha_alta

FROM usuarios;
```

Ejercicio

Ejecuta el siguiente script:

```
-- Creamos una nueva tabla llamada otros_usuarios exactamente igual a la tabla usuarios
SELECT id, dni, nombre, apellidos, email, descuento, fecha_alta INTO otros_usuarios
FROM usuarios;

SELECT * FROM otros_usuarios;

-- Borramos todas las filas de la tabla otros_usuarios
DELETE FROM otros_usuarios;

--Insertamos en la tabla otros_usuarios, que está vacía, las filas devueltas por la
instrucción SELECT
INSERT INTO otros_usuarios (id, dni, nombre, apellidos, email, descuento, fecha_alta)
SELECT id, dni, nombre, apellidos, email, descuento, fecha_alta
FROM usuarios;
```

4 Modificación de registros

La modificación de registros ya insertados en la tabla se realiza con la sentencia `UPDATE`, que tiene el siguiente formato:

```
UPDATE nombre_tabla

SET columna = valor [ {, columna = valor} ]

[ WHERE condiciones ]
```

Veamos algunos ejemplos:

- Actualizar una columna de una fila

```
UPDATE usuarios

SET nombre = 'Felipe'

WHERE id = 12;
```

- Actualizar varias columnas de una fila

```
UPDATE usuarios

SET nombre = 'Felipe', dni = '12365478H'

WHERE id = 15;
```

- Actualizar una columna de varias filas.
  Se actualizarán aquellas filas de la tabla pistas que cumplan la condición de la cláusula WHERE.

```
-- Incrementa el precio en un 10% de las pistas de tenis que tengan un precio
actual inferior a 20 €

UPDATE pistas

SET precio = precio + precio * 0.10

WHERE precio < 20 AND tipo = 'tenis';
```

- Actualizar una columna utilizando una subconsulta:

```
-- Reduce el precio de las pistas que no se han reservado todavía en un 10%

UPDATE pistas

SET precio = precio - precio * 0.1

WHERE id NOT IN (SELECT id_pista FROM reservas);
```

Cabe destacar que la ausencia de condiciones en una sentencia de actualización, ejecutaría dicha sentencia sobre todas las filas de la tabla.

5 Eliminación de registros

El borrado de filas de una tabla se efectúa con la sentencia `DELETE`, que tiene el siguiente formato:

```
DELETE FROM nombre_tabla

[ WHERE condiciones ]
```

Veamos algunos ejemplos:

- Elimina todos los usuarios

```
DELETE FROM usuarios;
```

- Borrar una fila estableciendo una condición

```
DELETE FROM pistas

WHERE id = 10;
```

- Borrar varias filas estableciendo varias condiciones

```
DELETE FROM pistas

WHERE tipo = 'baloncesto' OR codigo = 'BAL001';
```

- Borrar filas relacionando varias tablas

```
-- Elimina los usuarios que se dieron de alta antes de 2014

-- y aún no han reservado ninguna pista

DELETE FROM usuarios

WHERE id NOT IN (SELECT id_usuario FROM usuario_reserva)

AND fecha_alta < '2014-01-01';
```

Cabe destacar que la ausencia de condiciones en una sentencia de borrado, eliminaría todas las filas de la tabla.

```
-- Borraría todas las filas de la tabla usuarios

DELETE FROM usuarios
```